MusicXML Mapping Document (Simple Exporter)

This document describes how each field in the `Note`, `Rest`, `Measure`, and `Score` classes maps to MusicXML elements and attributes for a simple exporter that: - Does not use `<backup>` or `<forward>` (assumes events are sequential and fill the measure). - Supports note/rest durations up to sixteenth notes (lookup table). - Ignores velocity. - Uses a fixed `<divisions>` of 4 (so a sixteenth note = 1 XML duration unit). - Does not handle tuplets, multiple voices, key signatures, or lyrics.

---

1. Note → MusicXML `<note>`

Score Model Field	MusicXML Mapping	Details
`pitch_name` (e.g., `"C4"`)	`<pitch>` → `<step>`, `<alter>`, `<octave>`	Parse scientific pitch notation: • Letter (A-G) → `<step>` • Accidental (#/b) → `<alter>` (+1 for #, -1 for b, 0 for natural) • Octave number → `<octave>`
`midi` (0-127)	`<pitch>` (alternative)	If `pitch_name` unavailable, derive step/alter/octave from MIDI (MIDI 60 = C4).
`start_time` (beats)	**Position**	Not used; events are output in the order they appear in `Measure.events`. The exporter assumes events are already sorted by `start_time` and that there are no gaps (or gaps are ignored).
`duration` (beats)	`<duration>` (integer) + `<type>`	Lookup table: • 0.25 → sixteenth (`<duration>1</duration>`, `<type>16th</type>`) • 0.5 → eighth (`<duration>2</duration>`, `<type>eighth</type>`) • 1.0 → quarter (`<duration>4</duration>`, `<type>quarter</type>`) • 2.0 → half (`<duration>8</duration>`, `<type>half</type>`) • 4.0 → whole (`<duration>16</duration>`, `<type>whole</type>`) Where `<divisions>` = 4 (ticks per quarter note).
`velocity` (0.0-1.0)	**Ignored**	Not exported in this simple version.

Additional MusicXML elements for a note:
<voice> – *set to* 1.
<staff> – set to 1.

---

2. Rest → MusicXML <rest>

Score Model Field	MusicXML Mapping	Details
start_time (beats)	**Position**	Same as Note: ignore; output rests in order of Measure.events.
duration (beats)	<duration> (integer) + <type>	Same lookup table as Note (using <divisions> = 4).
(no pitch/ velocity)	<rest> element only	No <pitch> child.

Additional MusicXML elements for a rest:
<voice> – 1.
<staff> – 1.

---

3. Measure → MusicXML <measure>

Score Model Field	MusicXML Mapping	Details
number (int)	number attribute on <measure>	1-indexed.
events (List[Note∪Rest])	Sequence of <note> or <rest> elements	Events are output in the order they appear in the list (assumed sorted by start_time). No <backup>/<forward> inserted.
time_signature_top (int)	<attributes> → <time> → <beats>	Written once per measure if it differs from previous measure (or always for simplicity).
time_signature_bottom (int)	<attributes> → <time> → <beat-type>	Same as above.
(implicit)	<attributes> → <clef>, <key>, <divisions>	• <clef>: default treble clef (<sign>G</sign>, <line>2</sign>). • <key>: default C major (<fifths>0</fifths>). • <divisions>: fixed value 4.

MusicXML structure (per measure):

```
<measure number="{Measure.number}">
  <attributes>   <!-- include if first measure or time signature changed -->
    <divisions>4</divisions>
    <key>
      <fifths>0</fifths>
    </key>
    <time>
      <beats>{Measure.time_signature_top}</beats>
      <beat-type>{Measure.time_signature_bottom}</beat-type>
    </time>
    <clef>
      <sign>G</sign>
      <line>2</line>
    </clef>
  </attributes>
  <!-- events (notes/rests) in order -->
</measure>
```

---

4. Score → MusicXML `<score-partwise>`

Score Model Field	MusicXML Mapping	Details
`title` (str)	`<work>` → `<work-title>`	Top-level identification.
`composer` (str)	`<identification>` → `<creator>` (type="composer")	Part of `<identification>` block.
`measures` (List[Measure])	`<part>` → multiple `<measure>` elements	All measures belong to a single `<part>` (single-instrument score).
`tempo_bpm` (float)	`<direction>` → `<sound tempo="..."/>`	Placed at the start of the first measure. Metronome mark: `quarter = tempo_bpm`.

Overall MusicXML skeleton:

```
<score-partwise version="4.0">
  <work>
    <work-title>{Score.title}</work-title>
  </work>
  <identification>
    <creator type="composer">{Score.composer}</creator>
  </identification>
  <part-list>
    <score-part id="P1">
      <part-name>Instrument</part-name>
    </score-part>
  </part-list>
  <part id="P1">
```

```
    <!-- measures go here -->
  </part>
</score-partwise>
```

---

5. Lookup Table for Duration (beats → XML)

Beats	XML Duration (divisions=4)	MusicXML <type>
0.25	1	16th
0.5	2	eighth
1.0	4	quarter
2.0	8	half
4.0	16	whole

If a duration not in this table is encountered, the exporter should raise an error or round to the nearest supported value (for MVP, assume only these values appear).

---

6. Summary Table

Model	Field	MusicXML Destination
Note	pitch_name	<pitch> (<step>, <alter>, <octave>)
Note	midi	(alternative) pitch derivation
Note	start_time	Ignored (events ordered)
Note	duration	<duration> + <type> via lookup table
Note	velocity	**Ignored**
Rest	start_time	Ignored (events ordered)
Rest	duration	<duration> + <type> via lookup table
Measure	number	<measure number="...">
Measure	events	Ordered <note>/<rest> sequence
Measure	time_signature_top	<attributes>→<time>→<beats>
Measure	time_signature_bottom	<attributes>→<time>→<beat-type>
Score	title	<work>→<work-title>
Score	composer	

Model	Field	MusicXML Destination
		`<identification>`→`<creator type="composer">`
Score	measures	`<part>`→`<measure>` elements
Score	tempo_bpm	`<direction>`→`<sound tempo="...">` (quarter = BPM)

This mapping provides the simplest valid MusicXML exporter for the given constraints, suitable for exporting scores containing only whole, half, quarter, eighth, and sixteenth notes and rests.